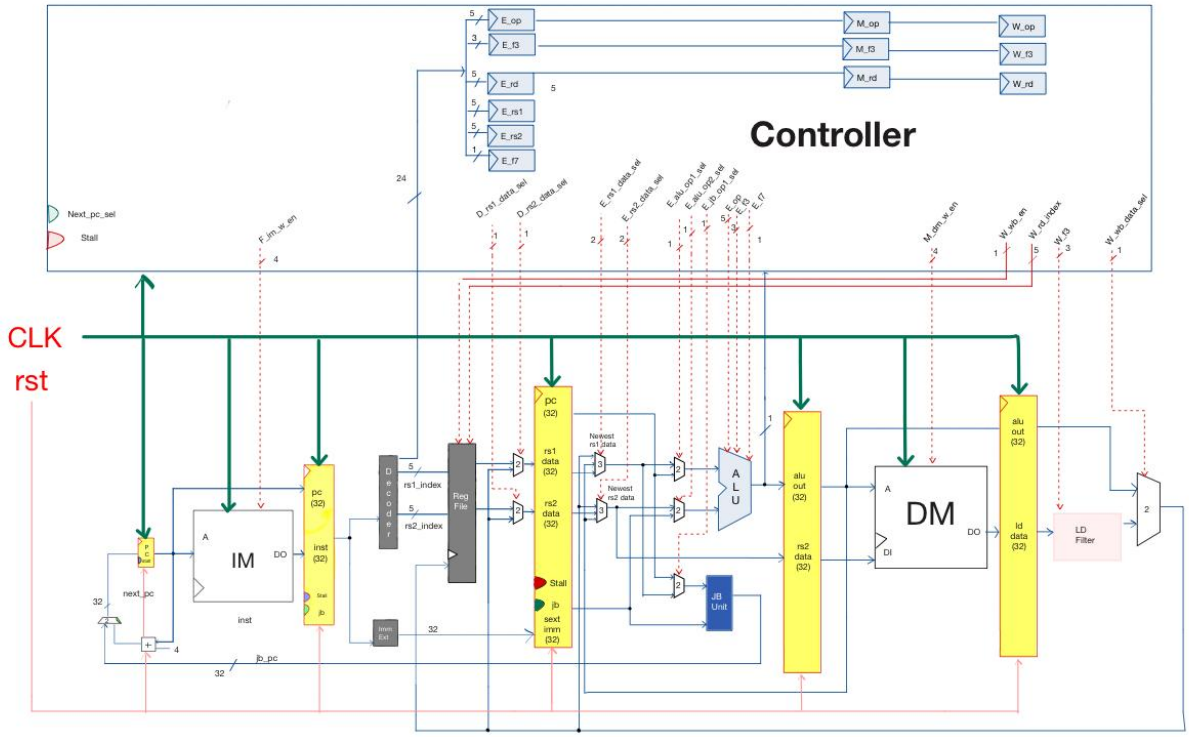
Lab8 結報

1. 實驗結果

A.CPU 架構圖



b. 執行結果截圖

Prog1:

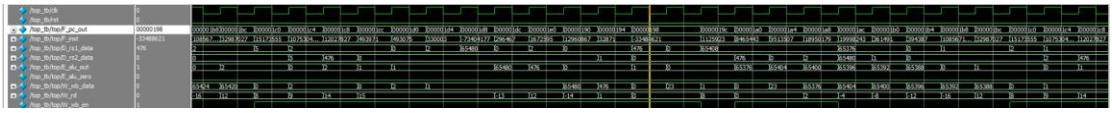
```
VSIM 21> run -all
#
# Done
#
# DM['h9000] = 00000005, pass
# DM['h9004] = 00000006, pass
# DM['h9008] = 00000007, pass
# DM['h900c] = 0000000b, pass
# DM['h9010] = 0000000c, pass
# DM['h9014] = 0000000d, pass
#
#
#
#
#                                        `;-.          ___,
#          ****************************    `.`\_...._/`.-"`
#          **                        **      \        /      ,
#          **                        **      /()    () \    .' `-._
#          **   Congratulations !!   **     |)  .    ()\ /   _.'
#          **                        **     \  -'-     ,; '. <
#          **                        **      ;.__     ,;|   > \
#          **   Simulation PASS!!    **     / ,    / ,  |.-'.-'
#          **                        **    (_/    (_/ ,;|.<`
#          **                        **      \    ,     ;-`
#          ****************************       >   \    /
#                                            (_,-''`> .'
#                                                 (_,'
#
#
# ** Note: $finish    : C:/computer organization/LAB8_TEST/top_tb.sv(97)
#    Time: 8795 ns  Iteration: 2  Instance: /top_tb
# 1
```

Prog0:

```
# DM['h9004] = ffffff8, pass
# DM['h9008] = 00000008, pass
# DM['h900c] = 00000001, pass
# DM['h9010] = 00000001, pass
# DM['h9014] = 78787878, pass
# DM['h9018] = 000091a2, pass
# DM['h901c] = 00000003, pass
# DM['h9020] = fefcfefd, pass
# DM['h9024] = 10305070, pass
# DM['h9028] = cccccccc, pass
# DM['h902c] = ffffffcc, pass
# DM['h9030] = ffffcccc, pass
# DM['h9034] = 000000cc, pass
# DM['h9038] = 0000cccc, pass
# DM['h903c] = 00000d9d, pass
# DM['h9040] = 00000004, pass
# DM['h9044] = 00000003, pass
# DM['h9048] = 000001a6, pass
# DM['h904c] = 00000ec6, pass
# DM['h9050] = 2468b7a8, pass
# DM['h9054] = 5dbf9f00, pass
# DM['h9058] = 00012b38, pass
# DM['h905c] = fa2817b7, pass
# DM['h9060] = ff000000, pass
# DM['h9064] = 12345678, pass
# DM['h9068] = 0000f000, pass
# DM['h906c] = 00000f00, pass
# DM['h9070] = 000000f0, pass
# DM['h9074] = 0000000f, pass
# DM['h9078] = 56780000, pass
# DM['h907c] = 78000000, pass
# DM['h9080] = 00005678, pass
# DM['h9084] = 00000078, pass
# DM['h9088] = 12345678, pass
# DM['h908c] = ce780000, pass
# DM['h9090] = fffff000, pass
# DM['h9094] = fffff000, pass
# DM['h9098] = fffff000, pass
# DM['h909c] = fffff000, pass
# DM['h90a0] = fffff000, pass
# DM['h90a4] = fffff000, pass
# DM['h90a8] = 13579d7c, pass
# DM['h90ac] = 13578000, pass
# DM['h90b0] = fffff004, pass
#
#
#
#
#                                        `;-.          ___,
#          ****************************    `.`\_...._/`.-"`
#          **                        **      \        /      ,
#          **                        **      /()    () \    .' `-._
#          **   Congratulations !!   **     |)  .    ()\ /   _.'
#          **                        **     \  -'-     ,; '. <
#          **                        **      ;.__     ,;|   > \
#          **   Simulation PASS!!    **     / ,    / ,  |.-'.-'
#          **                        **    (_/    (_/ ,;|.<`
#          **                        **      \    ,     ;-`
#          ****************************       >   \    /
#                                            (_,-''`> .'
#                                                 (_,'
#
#
# ** Note: $finish    : C:/computer organization/LAB8_TEST/top_tb.sv(97)
#    Time: 6965 ns  Iteration: 2  Instance: /top_tb
# 1
```

B.波形圖解釋，以下使用 prog1

1.流程解釋



本圖展示了指令 addi sp, sp, -32 如何正確通過 CPU 的五個階段。此指令將暫存器 sp (x2) 的值減去 32。

詳細訊號變化分析：

Fetch Stage (T):

觀察訊號 F_pc_out 為 00000198。

CPU 正在從指令記憶體位址 0x198 取出指令。根據 Dump 檔，此處指令為 addi sp, sp, -32 。

訊號 F_inst：顯示為負數，對應 hex 0xFE010113。

Decode Stage:後 1 個 Cycle (T+1)

觀察訊號 D_rs1_data 變為 65408 十進位。

指令進入解碼階段。

rs1 是 sp (x2)此時 sp 的值為 65408 (十六進位的 0xFF80)，這是堆疊指標的當前位置。

Execute Stage:後 2 個 Cycle (T+2)

觀察訊號 E_alu_out 變為 65376 (十進位)。ALU 執行加法運算：rs1 + immediate

計算過程：65408 + (-32) = 65376 證明 ALU 功能正常。

訊號 E_alu_zero：為 0，因為結果不為零。

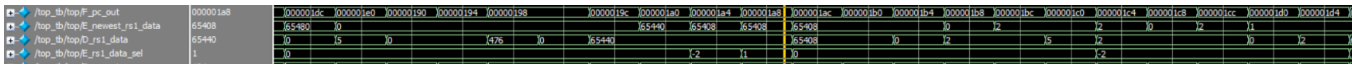
Writeback Stage:後 4 個 Cycle (T+4)

觀察訊號：W_wb_data 變為 65376，W_rd 變為 2，W_wb_en 為高電位。計算結果寫回 Register File 寫入值(W_wb_data)：65376 (正確的計算結果)。W_rd：2 對應 sp。寫入 enable

W_wb_en：1，表示允許寫入。

結論：波形圖證實了 I-Type 指令能夠正確讀取 Source Register，透過 ALU 正確計算 sp - 32，並在寫回階段將正確的結果更新至 sp，證明 CPU 基礎資料路徑與控制邏輯運作正常。

2. Data Forwarding



Hazard:

D_rs1_data 為 65440 (十進位)。這是從 Register File 讀出來的舊值。因為此時 addi 指令還在 WB Stage 還沒真正把新值 65408 寫進 Register File。如果 CPU 直接用這個值計算，記憶體位址就會算錯。

Forwarding Control:

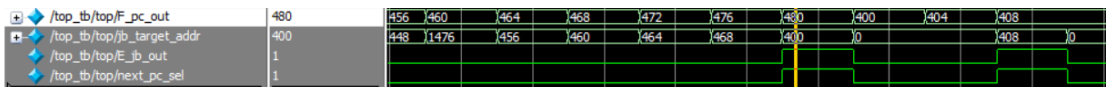
E_rs1_data_sel 為 1 這 Forwarding Unit 發出的訊號這表示偵測到了相依性，強制 Mux 忽略舊的暫存器值，改為選擇來自 WB 的資料。

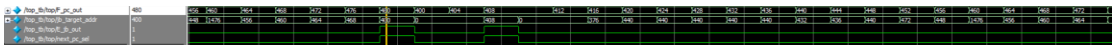
成功取得 Result:

E_newest_rs1_data 為 65408 這是經過 Mux 選擇後的正確新值即使 D_rs1_data 是 65440，ALU 最終拿到的 E_newest_rs1_data 卻是正確的 65408，證明這是前一個指令 addi 的計算結果。

波形圖顯示，Forwarding Unit 正確偵測到此 Hazard，將 E_rs1_data_sel 拉升為 1，啟動 Data Forwarding

3. Branch/Jump

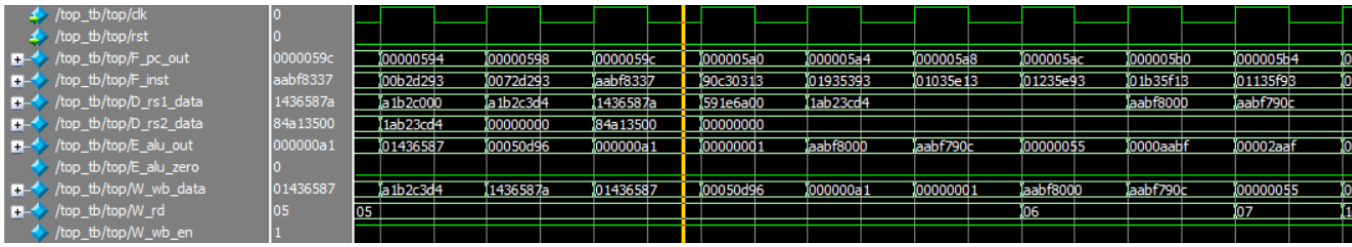




波形圖捕捉到了 mergeSort 進行遞迴呼叫。當位於 0x1D8(472dec)的 JAL 指令進入 Execute Stage 時，JB Unit 正確計算出目標位址為 400(十進位，同時將跳轉控制訊號 E_jb_out 拉升為 High。此動作觸發 next_pc_sel 訊號，指示 PC Mux 選擇跳 jb_target_addr。觀察 F_pc_out 訊號，PC 由原本 pipeline 預取的 480，在下一個 Clock Cycle 立即突變為 400。此結果證實了 CPU 成功執行跳轉，且 pipeline 的 Control Path 與 PC 更新邏輯運作正確

C. 波形圖解釋，以下使用 prog0

1. 流程解釋

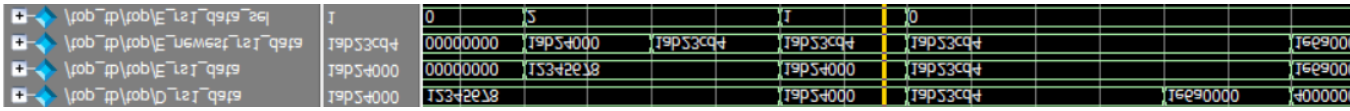


這是在執行 srli 與 srai 指令時的 pipeline 狀態

當波形圖停留在黃色時間線時，**F 階段**正在抓

取位址 5a0 的 addi，與 main.dump 完全吻合；同時，**D 階段**正對前一個週期的 lui 位址 59c 進行解碼與立即數處理；而 **E 階段**則呈現了再前一條指令的運算結果 0xa1，完整展示了各指令在流水線中依序遞進的執行過程。

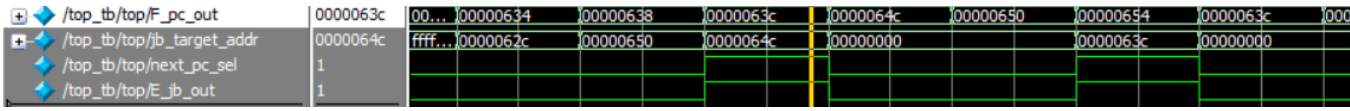
2. Data forwarding



本圖展示了流水線中 **Read-After-Write Hazard** 的處理機制。時間點於 Consumer Instruction 的執行階段 PC 0x1A8，該指令需要讀取 rs1 的數值。由於前一條指令 addi sp, sp, -32 尚未完成寫回階段，從 Register File 讀出的 D_rs1_data 仍為舊值 65440。此時 Forwarding Unit 偵測到資料相依性，將控制訊號 E_rs1_data_sel 拉高為 1。此動作切換了 ALU 前端 Mux，使其忽略舊值，轉而接收由 pipeline 後方的最新計算結果。波形顯示

E_newest_rs1_data 成功更新為 65408(65440 - 32)。這證實了運作正常，CPU 無需插入 Bubble 即可在 pipeline 中獲取正確 data。

3. Branch/Jump



波型圖 show JALR 流程，該指令位於 PC 0x63c。根據波形數據，在 E-Stage，ALU/JB Unit 依據指令 jalr t1, 4(t1)進行運算將 t1 值加上偏移量 4，得出跳轉目標位址 jb_target_addr 為 0x64c。同時，控制訊號 E_jb_out 與 next_pc_sel 拉高，PC Mux 更新 PC。在下一個 cycle，F_pc_out 成功由 0x63c 跳轉至目標 0x64c。此結果證實 CPU 能正確處理基底位址+偏移量的跳轉。